LAPORAN PRAKTIKUM PEMROGRAMAN BACKEND LANJUT — TM 11

Topik: Background Jobs, Asynchronous Processing, dan Transactional Outbox

Nama: MOHAMMAD ANHAZ ABDILAH

NIM: 434241102

Program Studi: D4 Teknik Informatika — Fakultas Vokasi Universitas Airlangga

Repository: AanFLenrir/latihan-fiber (Branch: main)

Lingkungan: Windows 11 64-bit, Go 1.24, Fiber v2.52, PostgreSQL 18, MongoDB v2, Memurai v4.1.2 (Redis), Asynq v0.26.0

Tanggal Pengujian: 01 November 2026

BAGIAN D.1 — BERITA ACARA PERBAIKAN SEMBILAN KESALAHAN

Sesuai dengan ketentuan *Modul 11 Edisi Puzzle*, kode modul memuat sembilan kesalahan yang sengaja ditanamkan. Seluruh sembilan kesalahan telah diinvestigasi, dibuktikan akar masalahnya, diperbaiki, dan divalidasi melalui pengujian compiler maupun pengujian fungsional database.

No.	Lokasi (File, Fungsi)	Gejala yang Diamati	Uji Pengungkap	Akar Masalah	Perbaikan yang Diterapkan	Status
1	`helper/mailer.go` `NewLogMailer`	Compiler error: `LogMailer does not implement Mailer (method Send has pointer receiver)`.	Compiler (Langkah 4)	`NewLogMailer` mengembalikan struct value `LogMailer{...}`, padahal method `Send` memiliki pointer receiver (`m *LogMailer`). State counter dan mutex juga wajib pointer agar terbagi antar goroutine.	Mengembalikan pointer `&LogMailer{...}`.	TERVERIFIKASI
2	`app/service/outbox_relay.go` `RelayOnce`	Saat Redis mati, event yang gagal di-publish tetap ditandai `published_at = NOW()` di PostgreSQL sehingga notifikasi hilang permanen saat Redis hidup kembali.	Uji C.2 (Redis mati)	`ids = append(ids, e.ID)` dieksekusi sebelum `r.publish(ctx, e)`. ID event yang gagal tetap masuk ke slice yang ditandai published oleh `MarkPublished`.	Pindahkan `ids = append(...)` setelah evaluasi keberhasilan `r.publish`.	TERVERIFIKASI
3	`app/service/course_service.go` `Enroll`	Dual-write atomicity violation: jika langkah 5 (pencatatan enrollment) gagal karena duplicate (409), outbox event tetap tersimpan dan email tetap terkirim.	Uji C.1 (Percobaan 2 duplicate)	Pemanggilan `s.outbox.Insert(...)` menggunakan koneksi pool langsung tanpa membungkusnya dalam transaksi aktif (`tx`).	Ubah pemanggilan menjadi `s.outbox.WithTx(tx).Insert(...)`.	TERVERIFIKASI
4	`app/service/course_service.go` `Drop`	Mahasiswa yang membatalkan kelas (Drop) menerima email "KRS: berhasil terdaftar di..." alih-alih konfirmasi pembatalan.	Uji C.1 (Pemeriksaan /mine)	Event type yang dicatat pada operasi Drop adalah `EventEnrollmentCreated` bukannya `EventEnrollmentDropped`.	Ganti event type menjadi `model.EventEnrollmentDropped`.	TERVERIFIKASI
5	`app/service/notification_worker.go` `HandleSend`	Compiler error: `cannot use worker.HandleSend as asynq.HandlerFunc in mux.HandleFunc`.	Compiler (Langkah 8)	Parameter task pada method handler didefinisikan sebagai struct value `t asynq.Task`, padahal signature `asynq.HandlerFunc` mensyaratkan pointer `*asynq.Task`.	Ubah tipe parameter menjadi `t *asynq.Task`.	TERVERIFIKASI
6	`app/service/notification_worker.go` `HandleSend`	Task dengan kegagalan permanen (domain `.invalid`) di-retry berulang kali hingga 5 kali alih-alih langsung masuk `archived`.	Uji C.4 & Eksperimen D.3	Error diformat dengan `%v: %w` (`asynq.SkipRetry` diformat dengan `%v`), sehingga `errors.Is` di internal Asynq tidak mengenali `SkipRetry`.	Ganti format pembungkusan error menjadi `fmt.Errorf("%w: %v", asynq.SkipRetry, err)`.	TERVERIFIKASI
7	`app/service/notification_worker.go` `HandleSend`	Race condition pada claim idempotensi saat beberapa goroutine worker berjalan konkuren. Notifikasi ganda bisa terkirim.	Uji C.5 (Salinan task)	Method `Claim` dipanggil langsung melalui `w.notifications.Claim(...)` tanpa mengikat transaksi database `WithTx(tx)`.	Gunakan `w.notifications.WithTx(tx).Claim(...)` di dalam closure `WithTxOnce`.	TERVERIFIKASI

No.	Lokasi (File, Fungsi)	Gejala yang Diamati	Uji Pengungkap	Akar Masalah	Perbaikan yang Diterapkan	Status
8	`config/queue.go` `NewWorkerServer`	Worker Asynq tidak pernah mengambil dan memproses task apa pun dari Redis. Task menggantung selamanya di antrean pending.	Uji C.1 (Relay vs Worker)	Konfigurasi antrean pada `Queues` memetakan nama jenis task (`"notification:send"`) bukannya nama queue (`"notifications"`).	Ubah key map antrean menjadi `Queues: map[string]int{model.QueueNotifications: 1}`.	TERVERIFIKASI
9	`app/service/outbox_relay.go` `publish`	Compiler error: `cannot use job (variable of type NotificationJob) as []byte value in argument to asynq.NewTask`.	Compiler (Langkah 7)	Fungsi `asynq.NewTask` membutuhkan payload bertipe `[]byte`, sedangkan modul langsung mengoper struct `model.NotificationJob`.	Lakukan `json.Marshal(job)` terlebih dahulu lalu oper byte slice ke `asynq.NewTask`.	TERVERIFIKASI

BAGIAN D.2 — NOTIFIKASI PERUBAHAN KUOTA UNTUK SELURUH PESERTA

Fitur ini menambahkan pengiriman notifikasi massal (*fan-out*) kepada seluruh peserta kelas yang telah terdaftar ketika Admin mengubah kuota kelas melalui endpoint `PATCH /api/v1/courses/:id`.

1. Mengapa Operasi Perubahan Kuota dan Pencatatan Outbox Harus Atomik?

Sebelum Modul 11, `UpdateQuota` hanya menjalankan satu pernyataan SQL dengan optimistic locking. Penambahan notifikasi outbox menjadikannya dua operasi: (1) update kuota pada tabel `courses`, dan (2) insert event `course.quota_changed` pada tabel `outbox`. Kedua pernyataan ini **wajib berada di dalam satu transaksi database (WithTx)** karena:

- Jika kuota berhasil diubah tetapi pencatatan outbox gagal (karena koneksi terputus atau server mati), perubahan kuota tersimpan namun mahasiswa tidak pernah menerima pemberitahuan (*lost event*).
- Jika outbox tersimpan mandiri tetapi update kuota gagal (karena konflik versi `version_conflict` atau kuota di bawah jumlah peserta `quota_below_enrolled`), email perubahan kuota palsu akan terkirim padahal kuota kelas sebenarnya tidak berubah.

2. Migration 009 (`migrations/009_quota_notifications.sql`)

Pada pendaftaran reguler, relasi antara event outbox dan notifikasi adalah 1-to-1 (satu event menghasilkan satu notifikasi untuk satu mahasiswa). Namun pada perubahan kuota, satu event outbox menghasilkan banyak notifikasi (1-to-N fan-out) untuk seluruh peserta kelas. Oleh karena itu, constraint lama `UNIQUE (outbox_id)` diganti menjadi **composite uniqueness `UNIQUE (outbox_id, user_id)`**.

```
-- migrations/009_quota_notifications.sql
ALTER TABLE notifications DROP CONSTRAINT IF EXISTS notifications_outbox_unique;
ALTER TABLE notifications ADD CONSTRAINT notifications_outbox_user_unique UNIQUE (outbox_id, user_id);
```

Keunggulan desain ini: Idempoten per mahasiswa. Jika pengiriman ke mahasiswa ke-3 gagal sementara (temporary failure), transaksi rollback hanya membatalkan claim mahasiswa ke-3. Saat Asynq me-retry task, mahasiswa ke-1 dan ke-2 yang sudah berhasil dikirim akan di-skip (`already_sent = true`) karena pasangan `(outbox_id, user_id)` mereka sudah terkunci di database.

3. Potongan Kode Penting D.2

```
// app/service/course_service.go (UpdateQuota dalam WithTx)
var updated model.Course
err := s.tx.WithTx(ctx, func(tx repository.DBTX) error {
    var err error
    updated, err = s.courses.WithTx(tx).UpdateQuota(ctx, id, req.Quota, req.Version)
    if err != nil {
        return err
    }
    return s.outbox.WithTx(tx).Insert(ctx, model.EventCourseQuotaChanged, model.CourseQuotaChangedEvent{
        CourseID: updated.ID, CourseCode: updated.Code, CourseName: updated.Name, Quota: updated.Quota,
    })
})
```

```
// app/service/notification_worker.go (Pemrosesan Fan-Out Idempoten)
func (w *NotificationWorker) handleQuotaChanged(ctx context.Context, job model.NotificationJob) error {
    var event model.CourseQuotaChangedEvent
    json.Unmarshal(job.Payload, &event)
    students, _ := w.courses.FindEnrolledStudents(ctx, event.CourseID)

    for _, student := range students {
        msg := BuildQuotaMessage(event, student)
        alreadySent := false
        err := w.tx.WithTxOnce(ctx, func(tx repository.DBTX) error {
            claimed, err := w.notifications.WithTx(tx).Claim(ctx, model.Notification{
                OutboxID: job.OutboxID, UserID: student.ID, Channel: "email",
                Subject: msg.Subject, Body: msg.Body,
            })
            if err != nil || !claimed {
                if !claimed { alreadySent = true }
                return err
            }
            return w.mailer.Send(ctx, msg)
        })
        if err != nil { return err } // Temporary error -> retry by Asynq
    }
    return nil
}
```

4. Hasil Pengujian Fungsional D.2 (Kelas IF399 dengan 5 Peserta & MAILER_FAIL_EVERY=3)

- Admin mengubah kuota kelas `IF399` (Kelas Rebutan) dari 5 menjadi 10 melalui `PATCH /api/v1/courses/13` (Status: `200 OK`).
- Event outbox ID 26 tercatat dengan type `course.quota_changed` dan status `published = true`.
- Dengan `MAILER_FAIL_EVERY=3`, kegagalan sementara terjadi pada attempt ke-3 dan ke-6. Worker otomatis di-retry oleh Asynq. Mahasiswa yang sudah sukses terkirim tidak menerima email ganda.
- Seluruh 5 mahasiswa (`mhs001` s.d. `mhs005`) menerima notifikasi di database dan tercatat di `logs/mail.log` dengan subjek tepat: `KRS: kuota IF399 Kelas Rebutan berubah menjadi 10`.
- Saat alat `tools/redeliver` dijalankan untuk menginjeksi 2 salinan task outbox 26, seluruh 5 mahasiswa berstatus `already_sent: true` dan 0 email ganda terkirim.

BAGIAN D.3 — BUKTI PENGUJIAN DAN PERCOBAAN PEMBANDING

1. Ringkasan Hasil Pengujian Penerimaan Bagian C

Skenario	Kondisi Pengujian	Hasil Aktual	Status
C.1 Jalur Normal	Mahasiswa mendaftar (201), duplikat (409), drop (204), daftar (201), swap (200).	Outbox tepat 4 baris, 4 notifikasi di `/notifications/mine`, 4 baris di `logs/mail.log`.	PASS
C.2 Redis Mati	Redis dimatikan, daftar ke kelas 5, 6, 7. Cek admin queue, lalu nyalakan Redis kembali.	Ketiga pendaftaran 201 (2–17 ms). `outbox_pending: 3` saat mati. Setelah hidup, `outbox_pending: 0` dan 7 notifikasi terkirim.	PASS
C.3 Temporary Failure	`MAILER_FAIL_EVERY=3`, daftar ke 6 kelas. Menunggu siklus retry Asynq.	DB notifications = 6, `/notifications/mine` = 6, `logs/mail.log` = 6. Log mencatat minimal 2 kali `task_failed` sementara.	PASS
C.4 Permanent Failure	Akun `salahalamat` dengan email domain RFC 2606 `.invalid` mendaftar ke kelas 2.	KRS sukses 201. Task gagal permanen, `archived` bertambah 1, `retry: 0`. Attempt berhenti di 1/6 dengan `SkipRetry`.	PASS
C.5 Idempotency	Injeksi 5 salinan task via `redeliver` + 1 task relay (total 6 salinan konkuren).	DB notifications tepat 1, `logs/mail.log` tepat 1 baris. Log worker: 1 `already_sent: false` dan 5 `already_sent: true`.	PASS
C.6 Graceful Shutdown	`MAILER_DELAY_MS=5000`, daftar ke kelas 9, kirim SIGINT/Ctrl+C saat pengiriman berjalan.	Worker menuntaskan in-flight task, mencatat `notification_processed`, lalu proses berhenti rapi tanpa memutus koneksi.	PASS

2. Percobaan Pembanding Terkontrol (Controlled Counter-Experiment)

Untuk membuktikan keabsahan alat uji, **Bug 6** (kesalahan verb format pada `SkipRetry`) dikembalikan ke kode:

Kondisi A: Kode Salah (Bug 6 Dikembalikan)

```
// app/service/notification_worker.go
if errors.Is(err, helper.ErrPermanentDelivery) {
    return fmt.Errorf("%v: %w", asynq.SkipRetry, err) // Verb %v menyebabkan SkipRetry tidak di-wrap
}
```

Hasil Eksekusi Uji C.4:
Compiler `go build ./...` lolos tanpa peringatan (membuktikan bug lolos compiler). Namun saat pengujian dijalankan:

```
Kondisi akhir /admin/queue: retry: 1, archived: 1 (archived tidak bertambah, retry bertambah)
[Log Worker]:
{"level":"WARN","msg":"task_failed","attempt":"1/6","error":"skip retry for the task:..."}
{"level":"WARN","msg":"task_failed","attempt":"2/6","error":"skip retry for the task:..."}
{"level":"WARN","msg":"task_failed","attempt":"3/6","error":"skip retry for the task:..."}
```

Kesimpulan Kondisi A: Task terus di-retry oleh Asynq karena `errors.Is(err, asynq.SkipRetry)` bernilai `false`. Pengujian C.4 **berhasil mendeteksi kesalahan tersebut**.

Kondisi B: Kode Benar (Perbaikan Diterapkan)

```
// app/service/notification_worker.go
if errors.Is(err, helper.ErrPermanentDelivery) {
    return fmt.Errorf("%w: %v", asynq.SkipRetry, err) // Verb %w membungkus SkipRetry dengan benar
}
```

Hasil Eksekusi Uji C.4:

```
Kondisi akhir /admin/queue: retry: 0, archived: 1 (archived bertambah tepat 1, retry bernilai 0)
[Log Worker]:
{"level":"WARN","msg":"task_failed","attempt":"1/6","error":"skip retry for the task: pengiriman gagal permanen: alamat salah@kampus.invalid tidak ditemukan"}
{"level":"WARN","msg":"Retry exhausted for task id=outbox-21","component":"asynq"}
```

Kesimpulan Kondisi B: Asynq langsung mengenali `SkipRetry` pada attempt 1/6 dan memindahkan task langsung ke antrean `archived` tanpa retry tambahan.

BAGIAN D.4 — ANALISIS SINGKAT

Pertanyaan 1: Dua Skenario Masalah pada Usulan Enqueue Langsung Setelah Commit dengan Retry di Handler

1. **Skenario Kehilangan Notifikasi (Dual Write Breakdown):**
   Ketika PostgreSQL selesai melakukan commit transaksi KRS, data pendaftaran telah resmi tersimpan di database. Jika tepat pada detik berikutnya server Redis mengalami crash, kehabisan memori, atau jaringan terputus, handler mencoba melakukan enqueue sebanyak 3 kali dan seluruhnya gagal. Karena tidak ada media persistensi lain tempat menyimpan task tersebut, notifikasi pendaftaran hilang permanen (*lost notification*). Hal yang sama terjadi jika proses web server mati mendadak (*crash/SIGKILL*) tepat setelah commit sebelum baris enqueue dieksekusi.
2. **Skenario Pembengkakan Waktu Respons Mahasiswa (Latency & Resource Starvation):**
   Jika Redis mengalami beban tinggi atau degradasi, setiap percobaan enqueue ke Redis akan menunggu hingga batas timeout (misal 2 detik per percobaan). Melakukan 3 kali retry di dalam handler HTTP memaksa browser mahasiswa menggantung (*blocking*) selama lebih dari 6 detik hanya untuk menunggu pengiriman email yang sebenarnya bukan syarat keabsahan KRS. Jika ribuan mahasiswa serentak mengakses portal, thread worker Fiber akan habis tertahan, menyebabkan portal web down (*cascading failure*).

Pertanyaan 2: Redis Restart Tanpa Persistence Saat 100 Task Pending

1. **Apa yang terjadi pada 100 notifikasi tersebut?**
   Ke-100 notifikasi tersebut hilang dari memori RAM Redis dan tidak akan pernah diproses oleh worker. Di sisi PostgreSQL, tabel `outbox` sudah menandai `published_at IS NOT NULL`, sehingga Outbox Relay tidak akan mengambilnya kembali.
2. **Deteksi Masalah Menggunakan Data PostgreSQL:**
   Membandingkan event outbox yang sudah berstatus `published_at IS NOT NULL` tetapi tidak memiliki baris padanannya di tabel `notifications` setelah jeda waktu tertentu (misal lebih dari 5 menit).
3. **Query Deteksi:**

```
SELECT o.id, o.event_type, o.payload, o.created_at, o.published_at
FROM outbox o
LEFT JOIN notifications n ON n.outbox_id = o.id
WHERE o.published_at IS NOT NULL
  AND n.id IS NULL
  AND o.published_at < NOW() - INTERVAL '5 minutes'
ORDER BY o.id ASC;
```

4. **Pemulihan Event:**

```
UPDATE outbox
SET published_at = NULL
WHERE id IN (
    SELECT o.id FROM outbox o
    LEFT JOIN notifications n ON n.outbox_id = o.id
    WHERE o.published_at IS NOT NULL
      AND n.id IS NULL
      AND o.published_at < NOW() - INTERVAL '5 minutes'
);
```

   Dengan mengembalikan `published_at` menjadi `NULL`, Outbox Relay pada putaran berikutnya akan otomatis menyeleksi kembali baris tersebut dan memasukkannya ulang ke Redis.
5. **Pencegahan Email Ganda:**
   Constraint `UNIQUE (outbox_id, user_id)` pada tabel `notifications` dan mekanisme `Claim` dengan klausa `ON CONFLICT DO NOTHING` menjamin bahwa jika ada task yang sebenarnya sudah sempat terkirim sebelum Redis mati, worker akan mendeteksi baris claim sudah ada (`already_sent = true`) dan langsung melewati pengiriman email.

Pertanyaan 3: Perbedaan Penggunaan `WithTxOnce` pada Worker vs `WithTx` pada CourseService

- **CourseService menggunakan `WithTx`:** Seluruh operasi dalam pendaftaran KRS adalah mutasi database internal PostgreSQL. Jika terjadi deadlock konkurensi atau serializable error, transaksi aman untuk diulang otomatis dari awal karena rollback database membersihkan seluruh efek samping tanpa jejak ke sistem luar.
- **Worker menggunakan `WithTxOnce`:** Di dalam worker, kita melakukan interaksi dengan sistem eksternal, yaitu mengirim email via SMTP / Mailer. Email yang telah terkirim ke jaringan luar tidak dapat dibatalkan (non-rollbackable). Jika worker menggunakan `WithTx` yang melakukan retry otomatis saat terjadi deadlock database, fungsi pengiriman email akan dipanggil ulang, mengakibatkan mahasiswa menerima email duplikat. Tanggung jawab retry diserahkan sepenuhnya ke Asynq dengan delay backoff terukur.
- **Risiko Jika Ditukar:** Jika worker memakai `WithTx`, mahasiswa berisiko menerima spam email konfirmasi berulang kali saat terjadi deadlock database. Jika `CourseService` memakai `WithTxOnce`, perebutan kursi kuliah antar mahasiswa akan langsung gagal tanpa retry, menurunkan keberhasilan transaksi mahasiswa saat sistem sibuk.

Pertanyaan 4: Urutan Email Tidak Dijamin pada Sistem Asinkron

1. **Kapan Berdampak Nyata bagi Pengguna?**
   Berdampak nyata ketika status pesan bersifat ketergantungan urutan (*state sequence dependency*). Misalnya mahasiswa mendaftar kelas lalu segera membatalkannya. Jika email pembatalan tiba lebih awal daripada email pendaftaran, mahasiswa dapat salah menyimpulkan bahwa ia saat ini terdaftar di kelas tersebut, padahal pembatalannya telah diproses di database.
2. **Mengapa Urutan Task Berbeda dengan Urutan Email?**
   Karena worker bekerja secara multi-threaded/konkruen (`WORKER_CONCURRENCY > 1`). Task daftar dan task keluar dieksekusi secara paralel oleh dua goroutine terpisah. Jika worker yang memproses task pendaftaran mengalami latensi jaringan SMTP atau terkena temporary retry, worker yang mengerjakan task keluar dapat menyelesaikan pengiriman lebih cepat.
3. **Solusi yang Masuk Akal:**
   Menyertakan nomor urut event / sequence ID (misal nilai `outbox_id` atau timestamp presisi) pada subjek dan isi email (misal: *"Status KRS Kelas IF102 [Revisi #4 - 18:49:33]"*). Di sisi web/mobile app, endpoint `/notifications/mine` selalu mengurutkan berdasarkan `outbox_id DESC`, bukan urutan pencatatan notifikasi.
4. **Konsekuensi Solusi Terhadap Performa:**
   Jika sistem dipaksa memproses strictly serial per mahasiswa menggunakan *single partition queue* atau *per-student mutex*, throughput pemrosesan background job akan menurun drastis dan kompleksitas arsitektur broker (seperti manajemen partisi Kafka atau antrean dinamis Redis) meningkat secara signifikan.